

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini Project Report

Of

Cloud Computing and DevOps Lab [CSE 3130]

WARDEN

A Verifiable Kubernetes Delivery Platform

SUBMITTED BY

Student	Registration number
Abhyuday Gupta	240905576
Aditya Chetan Thakur	240905588
Sankalp Arya	240905578
Shaurya Jain	240905598

Under the Guidance of

Mohandas Shenoy

School of Computer Engineering

Manipal Institute of Technology, Manipal

Academic Year 2026–27

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

CERTIFICATE

This is to certify that the mini project report entitled "Warden: A Verifiable Kubernetes Delivery Platform" is submitted by Abhyuday Gupta (240905576), Aditya Chetan Thakur (240905588), Sankalp Arya (240905578), and Shaurya Jain (240905598) for the Cloud Computing and DevOps Lab [CSE 3130], School of Computer Engineering, Manipal Institute of Technology, during the academic year 2026–27.

The report describes the work and results presented by the team. The signatures below are reserved for examination and approval.

Project guide	**Examiner**
Signature: _____	Signature: _____
Mohandas Shenoy	Name: _____
Date: _____	Date: _____

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION **4**

1.1 Context and 1.2 Objectives 4

1.3 Cloud services and scope 5

1.4 Research foundations and 1.5 Contribution 5

CHAPTER 2: METHODOLOGY **6**

2.1 Infrastructure and 2.2 Trusted build 6

2.3 Promotion, admission, and trust boundaries 7

2.4 Database and 2.5 Progressive delivery 8

2.6 Evaluation design 8

CHAPTER 3: RESULT ANALYSIS **9**

3.1 Experiment summary 9

3.2 Canary behavior 10

3.3 Delivery and rebuild evidence 11

3.4 Admission and database limits 11

3.5 Comparison and 3.6 Engineering findings 13

CHAPTER 4: CONCLUSION **14**

TEAM RESPONSIBILITIES 15

REFERENCES **16**

CHAPTER 1: INTRODUCTION

1.1 Context

We built Warden to bring service creation, software supply-chain checks, GitOps delivery, and automatic canary decisions into one Kubernetes platform. In five complete local trials, we created and deployed a service in a median of 8.6 minutes with no manual steps. In a separate 20-release canary experiment, all ten faulty releases aborted and all ten healthy releases promoted. We also completed five Azure rebuilds from recorded configuration.

We addressed a practical engineering problem: a successful build does not by itself establish what will run in a cluster or how a faulty release will be contained. Warden links an image digest to its build evidence, promotes that digest through a controlled Git change, and evaluates the release under traffic. We used four deliberately small sample services, books, loans, members, and notify, to test this delivery path repeatedly.

Our platform covers service creation, build and scanning, image signing, GitOps promotion, admission checks, progressive rollout, and a declarative PostgreSQL request. We ran it locally on kind and rebuilt and measured it on Azure Kubernetes Service (AKS). The same versioned configuration describes both environments.

1.2 Problem and objectives

A valid image signature alone does not establish that the expected repository, branch, or workflow produced an image. We therefore check the artifact digest and build identity, and keep the pipeline separate from cluster credentials. We set six measurable objectives:

Objective	Experiment
Create and deploy a service from one command	E1
Refuse artifacts that fail admission checks	E2
Abort a faulty canary without a manual rollback	E3
Provide a PostgreSQL database from a short request	E4
Measure delivery from deployment history	E5
Rebuild the environment from versioned configuration	E6

Table 1.1. Objectives and their evaluation experiments.

1.3 Cloud services and scope

We used AKS for managed Kubernetes; GitHub Actions for builds; GitHub Container Registry for immutable images; Microsoft Entra ID and GitHub OpenID Connect for workload identity; Sigstore for signing certificates and transparency entries; and Azure Storage for encrypted OpenTofu state. We used Argo CD for GitOps, Argo Rollouts for canaries, and Crossplane with CloudNativePG for databases. Together these services cover compute, registry, identity, storage, infrastructure as code, and delivery automation.

We implemented enforceable digest, registry, and Pod-hygiene rules, a cryptographic policy tested in Audit, and self-service databases through CloudNativePG. Azure Flexible Server support is in open review PRs and still needs a live provisioning and lifecycle test. In Chapter 3, we distinguish controlled Deny-mode tests from installed enforcement. We did not evaluate production availability, multiple regions, or runtime intrusion detection.

1.4 Research and technical foundations

We drew on several established ideas. Borg illustrates declarative workload management and automated recovery in a shared cluster. The in-toto work treats software integrity as a property of the steps that produce an artifact. Sigstore makes artifact signing practical through identity-backed certificates, short-lived keys, and public transparency records. SLSA defines interoperable provenance for describing where, when, and how an artifact was built. We apply these ideas through Kubernetes orchestration and signed build provenance. [1], [2], [3], [4]

We also drew on OpenGitOps principles for versioned desired state and continuous pull-based reconciliation. Google's canary-release guidance informed our partial deployment and rollout decisions. We used DORA to measure delivery throughput and instability. These sources informed our GitOps path, canary experiments, and delivery measurements. [5], [6], [7]

1.5 Project contribution

Our engineering contribution is the integration and evaluation of a complete delivery path. We built a service-creation CLI, a reusable build and signing workflow, constrained promotion, policy fixtures, experiment runners, and delivery-metric calculations. We tested invalid artifacts, injected faults, checked controller reconciliation, and rebuilt the environment. This let us inspect both working behavior and remaining gaps.

CHAPTER 2: METHODOLOGY

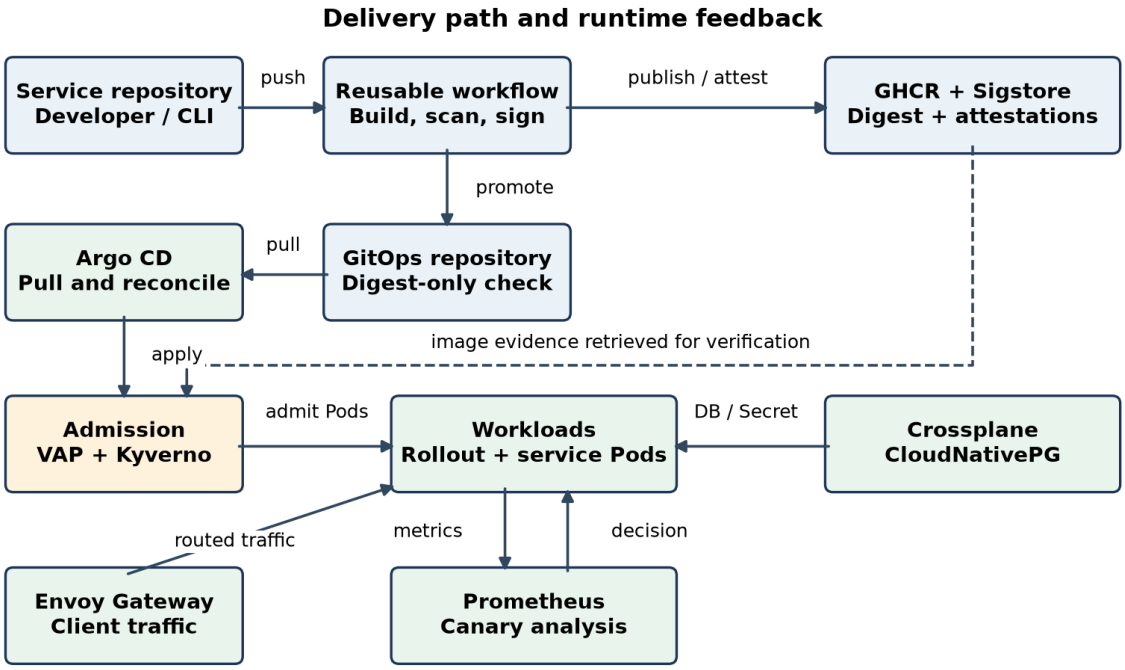


Figure 2.1. Warden delivery and cluster control flow.

2.1 Workspace and infrastructure

We separated infrastructure, GitOps state, admission policies, and platform tooling into four repositories. We use OpenTofu to prepare Azure resources. For local trials, we use kind in place of AKS while keeping the GitOps and admission paths comparable. Pinned component versions and image digests make our rebuilds repeatable. We timed cluster creation and destruction from a clean state, and retained failed attempts in the measurement records.

2.2 Service creation and trusted build

Our warden new command creates a service repository, a small application, a reusable delivery workflow, deployment manifests, and a catalog entry. A push calls our shared platform workflow, which builds and scans the image, signs its digest, attaches provenance, and opens a narrow GitOps pull request that changes the deployed digest. Only the signing job receives an identity token. No delivery job receives cluster credentials; Argo CD pulls the merged change. We pin third-party workflow actions to full commit hashes. Our artifact identity and production steps follow software supply-chain practices described by in-toto and Sigstore [2], [3].

2.3 Promotion, admission, and rollout

We require each promotion pull request to pass a digest-only check that permits one image-digest change. After that check passes, our promoter application can merge the change. Argo CD applies the desired state. Our native admission rules reject mutable tags, unapproved registries, and unsafe Pod settings. Kyverno checks signatures and provenance against the intended build identity in Audit mode. For our controlled attack experiment, we exercised the same policy in Deny mode. Argo Rollouts then shifts a small share of traffic to the new version and uses Prometheus metrics to promote or abort it. Envoy Gateway supplies the route. Our pull-based reconciliation and partial rollouts follow GitOps and canary principles [5], [6].

2.3.1 Trust boundaries and control design

We separated the repositories that build applications from the configuration that approves and deploys them. A service repository cannot rewrite the policy repository using its promotion credential. The build job cannot request a signing identity token; only the signing job has that permission, and it does not check out project code. This limits the credentials available to application build scripts. Our GitOps check also rejects a changed image name or an unrelated configuration edit.

Risk	**Implemented control**	**Evidence**
Mutable or substituted image	Digest references and registry allowlist	A01, A06
Untrusted build identity	Signed provenance binds digest, workflow, caller repository, and branch	A02–A08; Audit live
Build code accesses signing identity	Identity token restricted to isolated signing job	A10
Promotion changes unrelated configuration	Required digest-only check with no bypass	A11
Out-of-band policy drift	Argo CD reconciliation restores desired state	A12

Table 2.1. Selected risks, controls, and their evaluation coverage. Cryptographic admission is staged.

2.4 Database and measurement

We expose a namespaced database request through Crossplane and CloudNativePG. The composition creates a predictable connection Secret, so an application manifest can remain the same across kind and AKS. We recorded database readiness and a client connection. We compute delivery metrics from deployment events rather than transient cluster logs, and retain attack outcomes, canary timing, rebuilds, and verification latency with their experiment records.

2.5 Progressive delivery under measured traffic

We configure the rollout to advance through 5%, 25%, and 50% canary traffic before full promotion. Synthetic load supplies requests while Prometheus evaluates success rate and latency. We injected HTTP errors or extra latency to test both reliability and response time. An unsuccessful analysis aborts the canary, and a corrected release still passes through our normal build and promotion path. We distinguish time to abort from time to deploy a complete fix, following the canary approach described in [6].

We had to coordinate two controllers that manage traffic routing. Argo CD maintains the declared configuration, while Argo Rollouts changes route weights during a canary. Without an explicit field-ownership exception, reconciliation reset those weights and invalidated the experiment. Our final configuration lets Rollouts manage traffic weights while Argo CD reconciles the surrounding resource.

2.6 Evaluation design and reproducibility

We ran eight experiments covering service creation, security controls, canary behavior, database provisioning, delivery history, rebuilds, comparison with another platform, and verification overhead. We stored each observation with its procedure. We summarize successful rebuilds separately from failed attempts and retain the failures in the measurement records. We cross-check release metrics against the experiment runner so an incorrect event count does not become a performance claim. Our metric definitions follow DORA's five measures of delivery performance [6].

We used kind for affordable repeated local trials and AKS to test cloud provisioning, reconciliation, and policy behavior on managed Kubernetes. We label results by environment because a successful local series does not establish identical cloud behavior. Our small samples and deliberate fault injection limit the conclusions: these experiments demonstrate the recorded behavior, not a production reliability rate.

CHAPTER 3: RESULT ANALYSIS

3.1 Experiment summary

ID	Measured result	Interpretation
E1	5 full kind runs; median 8.6 min; 0 manual steps	One-command local path shown; Azure and hand-timed baseline open.
E2	15/15 security checks passed on each environment	Admission, workflow, promotion, and reconciliation controls; image policy audits live.
E3	App-only: 4/4 faults aborted; 12/12 healthy promoted	Median abort 147.6 s; faulty request share median 0.39%.
E4	Database ready: kind 804 s cold, 21 s warm; AKS 138 s	Same request and Secret shape; both backed by CloudNativePG.
E5	App-only batch: 20 deployments, 5 per service	Median lead time 12.5 min; failure and rework 20%; recovery 12.6 min.
E6	Local 11/12 rebuilds; Azure 5 successful rebuilds	Azure success median 22 min, with 4 failed attempts documented.
E7	One run each against idpbuilder	Warden used 8.8 GiB; idpbuilder 2.2 GiB and 8.5 min to first app.
E8	Warm verification 14–15 s per Pod	Similar on kind and AKS; cost is inside the verification path.

Table 3.1. Recorded results; the sample and scope are stated for each experiment.

Our five complete service-creation runs finished below the 15-minute target. We excluded the resumed books run from this sample. These results demonstrate our automated local path; our planned Azure series and hand-timed baseline remain open.

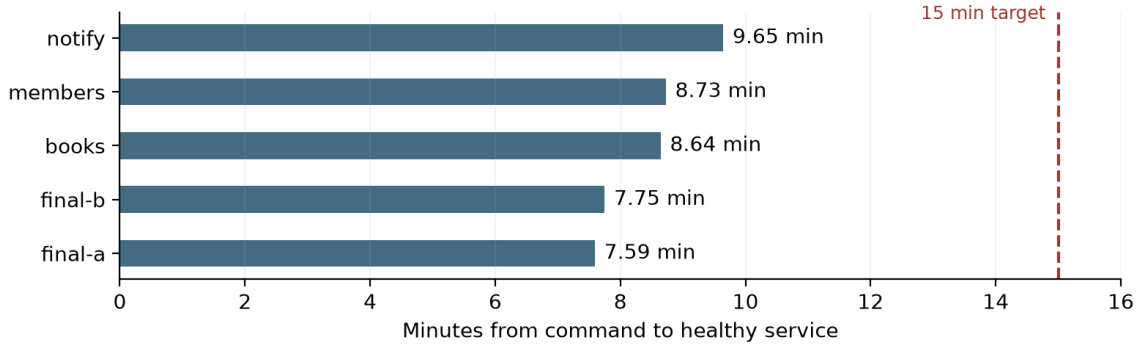


Figure 3.1. Complete local scaffolding trials, E1. Source: project experiment measurements.

3.2 Canary behavior

In our controlled rollout series, we tested ten deliberately faulty and ten healthy releases. All faulty releases aborted, and all healthy releases promoted. The median time from rollout start to abort was 147.2 seconds; the healthy promotion median was 471.6 seconds. The error trials exposed a median 0.397% of requests to the faulty canary, below the planned 0.5% share. These runs support the rollback claim for kind; we did not run a comparable Azure canary series.

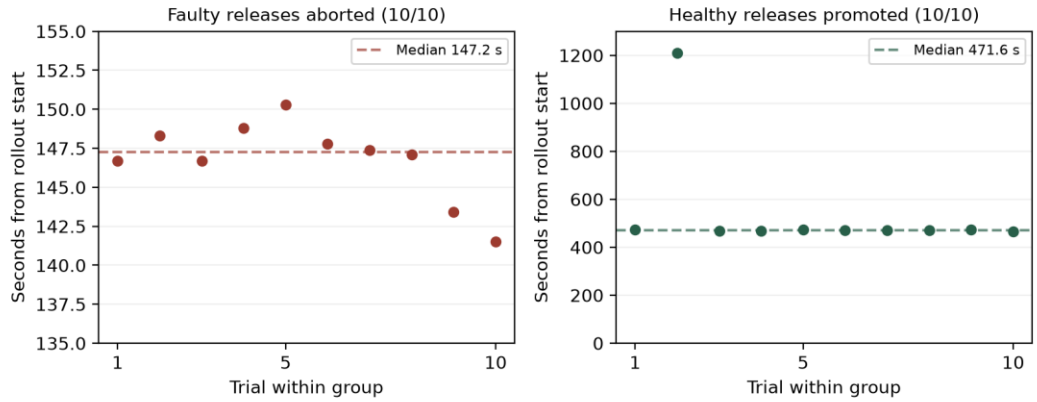


Figure 3.2. All 20 measured kind rollout outcomes. Panels use different vertical scales; dashed lines show medians. The healthy series includes the 1,209.5 s outlier.

We split the ten faulty trials between five error and five latency injections. In the error trials, the observed request failure share ranged from 0.325% to 0.509%; its median was 0.397%. The latency trials slowed a median 4.08% of service requests and produced no HTTP errors. We report these measures separately because they describe different failure modes.

We distinguish a quick abort from complete recovery. The median interval from abort to a corrected release being fully promoted was 759.2 seconds, including the new build and canary. One healthy trial took 1,209.5 seconds to promote; the other healthy trials took 466.2–474.1 seconds. We retain that outlier because the median alone would hide the spread.

3.3 Delivery and rebuild evidence

We ran two E5 batches. In the original 2026-10-26 batch, we used owner merges after the digest-only check. We planned its 20% failure rate, and its lead time excludes human review. Our separate 2026-10-27 app-only repeat records 20 deployments, five per service, with 16 successful and four injected-fault deployments. The promoter app merged each promotion after the required digest-only check, without an owner merge. Median change lead time was 12.5 minutes, change fail and rework rates were 20%, and median failed-deployment recovery was 12.6 minutes. This short synthetic batch does not represent normal team delivery frequency.

Our canary records for the app-only repeat show 12 healthy promotions and four injected-error releases aborted by the success-rate analysis, each followed by a healthy fix. Fault aborts took a median 147.6 seconds, with 0.39% of requests failing during the canary at a 5% traffic share. Abort-to-fixed median was 756.6 seconds. We retained one timed-out members CI capture and one healthy attempt that overlapped the members fault test as excluded records. The 20 deployments in the GitHub ledger agree with our measured set.

Service	Deployments	Median lead	Change fail	Median recovery
Books	5	12.8 min	20%	14.1 min
Loans	5	13.1 min	20%	11.8 min
Members	5	12.3 min	20%	12.6 min
Notify	5	12.5 min	20%	12.6 min

Table 3.2. Per-service DORA results in the 92-minute app-only experiment window.

Our calculated deployment frequency is about 250 per day in this short test window. We do not interpret that scripted rate as normal team cadence.

Service	GitOps pull request	Author and merger
notify	#97	warden-ccd-promoter
loans	#98	warden-ccd-promoter
members	#99	warden-ccd-promoter
books	#100	warden-ccd-promoter

Table 3.3. Verified v0.1.2 digest promotions on 27 October 2026.

We recorded 11 successful local rebuilds out of 12, with a median of 14.9 minutes and a 24.2-minute 90th percentile. We also completed five Azure rebuilds from an empty state with a median of 22 minutes. We retained and analyzed four failed Azure attempts instead of discarding them.

3.4 Admission and database limits

Our attack suite passed all 15 checks on kind and AKS, but those results do not represent 15 admission rejections. The suite includes one admitted positive control, eight direct admission denials, one ReplicaSet case whose child Pods were refused, and five checks covering earlier prevention, credential absence, signing permissions, promotion constraints, and drift recovery. In the drift test, we restored the policy after 1.4 seconds on kind and 1.6 seconds on AKS.

These results show controls at several points in our delivery path. They do not establish fail-closed operation of the installed image policy, which remains in Audit. Warm verification added about 14–15 seconds per Pod, and transient registry or TUF fetch errors interrupted some legitimate checks. We need another readiness run before changing enforcement.

Our measured database request worked in both environments and generated a usable connection URI; we used CloudNativePG on kind and AKS. Azure Flexible Server support is implemented in open infrastructure and GitOps pull requests but has not been merged or tested live. Our results establish portability of the request and Secret contract across clusters, not yet across database providers.

3.5 What the comparison establishes

We compared Warden with idpbuilder to understand the platform resource cost. In one recorded run, idpbuilder reached GitOps and a first application in 8.5 minutes using 2.2 GiB. Our recorded platform footprint was 8.8 GiB and included admission checks, canary analysis, observability, and database infrastructure. The bootstrap paths differed: the baseline deployed a pre-built public image, while our service-creation experiment included repository creation, building, signing, and promotion. This comparison describes scope and resource use; it is not an equal-feature speed benchmark.

3.6 Engineering findings

We had to resolve interactions between components to make several results possible. Our first promotion fix failed because the command-line client refused a blocked pull request before the server could apply the promoter's review bypass. Calling the merge endpoint with the expected head commit resolved the path while keeping the digest-only check mandatory. The four later service releases provide direct evidence that the revised path worked.

We also validated our measurement code. An early delivery-metric calculation counted only eight of twenty releases because older GitHub deployments became inactive and reconciliation announced a revision across multiple applications. We compared the event ledger against the experiment runner and found both errors. Our corrected result counts twenty releases and four injected failures.

We found another admission issue during database integration: operator-created Pods and initialization Jobs need narrowly scoped exceptions. A rule that appeared acceptable in Audit could have blocked every database in Deny mode. We tested legitimate workload classes alongside attacks to evaluate the policy. These experiments also exposed controller and measurement problems that a successful deployment alone would not have revealed.

CHAPTER 4: CONCLUSION

We demonstrated a working Kubernetes delivery platform whose key behaviors are traceable to recorded experiments. Our platform combines service scaffolding, an isolated signing workflow, digest promotion, admission controls, canary analysis, database requests, and repeatable infrastructure. We measured a local service-creation median of 8.6 minutes; in the app-only canary batch, all four injected faults aborted and all 12 measured healthy releases promoted; five Azure rebuilds recreated the platform in a median of about 22 minutes. We exercised the system beyond a single successful deployment.

We still need more work before claiming two promised outcomes are complete. Our installed cryptographic admission policy remains in Audit, even though a controlled Deny-mode suite passed. The Azure Flexible Server code is in review and still needs a live test. Our E1 sample is also smaller than planned. These limits separate features we tested from paths that are ready for production.

Our next steps are to remeasure image verification after a Kyverno fix, including legitimate Pods, registry failures, and timeout behavior before changing Audit to Deny; review and merge the Azure database changes, then test connection, deletion, and teardown; and expand our service-creation and canary measurements on AKS. We preserve the evidence for completed claims and identify the additional evidence these claims still need.

TEAM RESPONSIBILITIES

We confirmed the responsibility areas below as a team. They describe contribution areas, not a commit-by-commit authorship record; the repository history does not capture all of our design, review, evaluation, integration, and presentation work. Abhyuday Gupta led the architecture, core platform implementation, and cross-repository integration.

Member	Contribution area
Abhyuday Gupta	System architecture, trusted build workflow and CLI, cross-repository integration, and final evidence review
Aditya Chetan Thakur	Azure and local infrastructure, Crossplane database path, and rebuild evaluation
Sankalp Arya	Admission policy design and readiness, attack suite, threat model, and upstream policy reports
Shaurya Jain	GitOps delivery configuration, rollout and observability validation, and presentation/demo preparation

Table 4.1. Team contribution areas.

REFERENCES

[1] A. Verma, L. Pedrosa, M. R. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in Proc. EuroSys, 2015. Available: https://research.google/pubs/large-scale-cluster-management-at-google-with-borg/

[2] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, “in-toto: Providing farm-to-table guarantees for bits and bytes,” in Proc. 28th USENIX Security Symposium, 2019, pp. 1393–1410. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[3] Z. Newman, J. S. Meyers, and S. Torres-Arias, “Sigstore: Software Signing for Everybody,” in Proc. ACM SIGSAC Conference on Computer and Communications Security, 2022, pp. 2353–2367, doi: 10.1145/3548606.3560596.

[4] OpenSSF, “SLSA v1.0: Provenance.” Available: https://slsa.dev/spec/v1.0/provenance.

[5] OpenGitOps, “GitOps Principles,” version 1.0.0. Available: https://opengitops.dev/ (accessed Oct. 27, 2026).

[6] A. Warner and Š. Davidovič, “Canarying Releases,” in The Site Reliability Workbook, O’Reilly Media, 2018, ch. 16. Available: https://sre.google/workbook/canarying-releases/

[7] N. Harvey, “A history of DORA’s software delivery metrics,” DORA, Jan. 2, 2026. Available: https://dora.dev/insights/dora-metrics-history/ (accessed Oct. 27, 2026).